

C-DNN: An Energy-Efficient Complementary Deep-Neural-Network Processor With Heterogeneous CNN/SNN Core Architecture

Sangyeob Kim¹, Graduate Student Member, IEEE, Soyeon Kim², Graduate Student Member, IEEE, Seongyon Hong³, Graduate Student Member, IEEE, Sangjin Kim⁴, Graduate Student Member, IEEE, Donghyeon Han⁵, Graduate Student Member, IEEE, Jiwon Choi⁶, Graduate Student Member, IEEE, and Hoi-Jun Yoo⁷, Fellow, IEEE

Abstract—In this article, we propose a complementary deep-neural-network (C-DNN) processor by combining convolutional neural network (CNN) and spiking neural network (SNN) to take advantage of them. The C-DNN processor can support both complementary inference and training with heterogeneous CNN and SNN core architecture. In addition, the C-DNN processor is the first DNN accelerator application-specific integrated circuit (ASIC) that can support CNN–SNN workload division by using their magnitude–energy tradeoff. The C-DNN processor integrates the CNN–SNN workload allocator and attention module to find a more energy-efficient network domain for each workload in DNN. They enable the C-DNN processor to operate at the energy optimal point. Moreover, the SNN processing element (PE) array with distributed L1 cache can reduce the redundant memory access for SNN processing, resulting in a 42.2%–49.1% reduction. For high energy-efficient DNN training, the C-DNN processor integrates the global counter and local delta-weight (LDW) unit to eliminate power-consuming counters for a forward delta-weight generation. Furthermore, the forward delta-weight-based sparsity generation (FDWSG) is proposed to reduce the number of operations for training by 31%–79%. The C-DNN processor achieves an energy efficiency of 85.8 and 79.9 TOPS/W for inference with CIFAR-10 and CIFAR-100, respectively (VGG-16). Moreover, the C-DNN processor achieves ImageNet classification with state-of-the-art energy efficiency of 24.5 TOPS/W (ResNet-50). For training, the C-DNN processor achieves the state-of-the-art energy efficiency of 84.5 and 17.2 TOPS/W for CIFAR-10 and ImageNet, respectively. Furthermore, it achieves 77.1% accuracy for ImageNet training with ResNet-50.

Index Terms—Application-specific integrated circuit (ASIC), complementary deep neural network (C-DNN), convolutional neural network (CNN), deep learning, deep neural network, spiking neural network (SNN).

Manuscript received 9 May 2023; revised 12 July 2023, 13 September 2023, and 11 October 2023; accepted 1 November 2023. Date of publication 15 November 2023; date of current version 29 December 2023. This article was approved by Associate Editor Jae-Sun Seo. This work was supported by the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant funded by the Korean Government [Ministry of Science and ICT (MSIT)], Development of DRAM-Processing-in-Memory Chip for DNN Computing, under Grant 2023-0-00871. (Corresponding author: Hoi-Jun Yoo.)

The authors are with the School of Electrical Engineering, Korea Advanced Institute of Science and Technology, Daejeon 34141, South Korea (e-mail: hjyoo@kaist.ac.kr).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/JSSC.2023.3330483>.

Digital Object Identifier 10.1109/JSSC.2023.3330483

I. INTRODUCTION

RECENTLY, spiking-neural-network (SNN) application-specific integrated circuits (ASICs) [1], [2], [3], [4] were proposed to achieve low power consumption for always-on applications. For SNN, the input data are converted into spike train by using spike encoding such as rate encoding. Each neuron in the SNN accumulates weights whenever it receives an input spike to increase the membrane voltage, and when this value exceeds the threshold, a post-spike occurs, as shown in Fig. 1. Thus, it has event-driven characteristics in which computation is performed in the next neurons only when a post-spike occurs. In other words, SNN requires a small number of accumulations compared to convolutional neural network (CNN), and it can achieve low power consumption. In addition, many CNN-to-SNN conversion methods [5], [6], [7], [8], [9], [10] were proposed to overcome the accuracy limitation of SNN, as shown in Fig. 2. Previous SNN training ASICs [1], [2] cannot achieve high accuracy, but highly accurate SNN can be generated by converting the highly accurate CNN, which was trained with backpropagation (BP). For CNN-to-SNN conversion, the input feature map is converted into spike train, and the pre-trained weight is shared into SNN. Also, the ReLU function of the CNN is converted into integrated-and-firing neuron of SNN by finding the appropriate threshold. As a result, we can generate highly accurate SNN, which has less than 1% accuracy degradation compared to CNN.

However, there is a limitation of SNN for low-power processing, as shown in Fig. 3. The performance of SNN varies depending on distribution of input activation. In the case of distribution A, there are lots of small values, so spikes are sparsely generated by rate encoding. SNN requires additions as many time as the number of input spikes, and when the magnitude of the input is small, the SNN can consume less power. In contrast, in the case of distribution B, a dense spike pattern is generated because it has many large values. In contrast, the multiplier for CNN is composed of spatially arranged full adders, as shown in Fig. 4. In this case, even if the operand is small, a large number of full adders are operated, and the change in computational power

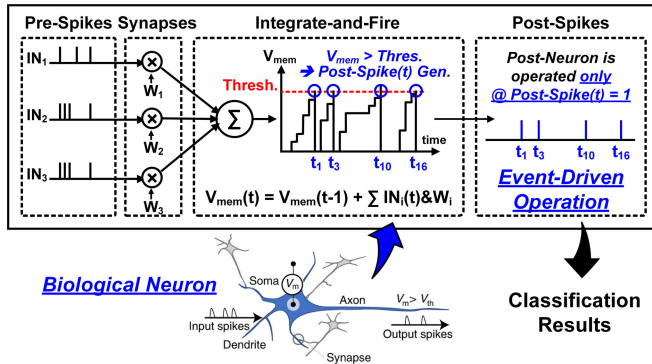


Fig. 1. Operation of the SNN.

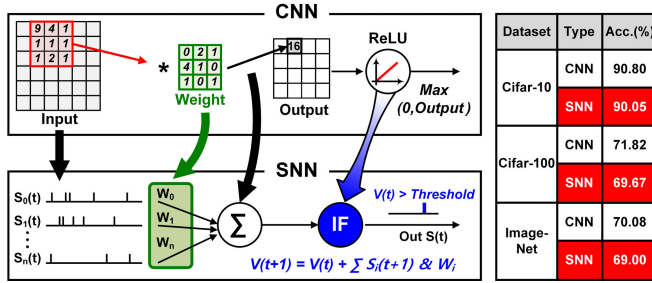


Fig. 2. CNN-to-SNN conversion algorithm.

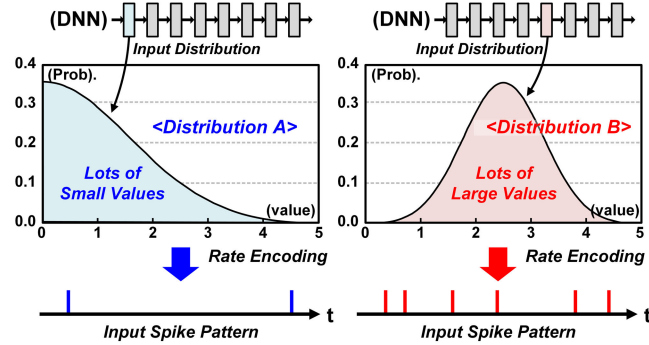


Fig. 3. Limitation of SNN for low-power processing.

consumption for magnitude is smaller than that of SNN. Therefore, there is a complementary characteristic between CNN and SNN. For large amount of small data, SNN with low computational power consumption is useful. Also, for a lot of large-magnitude data, CNN with low-power variation is useful.

Fig. 5 shows the complementary characteristics of CNN and SNN at neural network inference. Each layer of the neural network has a different input distribution. Layer 1 has a wide input distribution, so low-power SNN processing is not possible in this layer due to low spike sparsity. Conversely, layer 3 has a narrow input distribution, so low-power SNN processing is possible due to high spike sparsity. In other words, the optimal network domain varies layer-by-layer at DNN inference. In DNN training, there is also complementary characteristic in CNN and SNN, as shown in Fig. 6. CNN can achieve high accuracy by BP training, but it requires heavy computation due to iterative BP and gradient generation (GG).

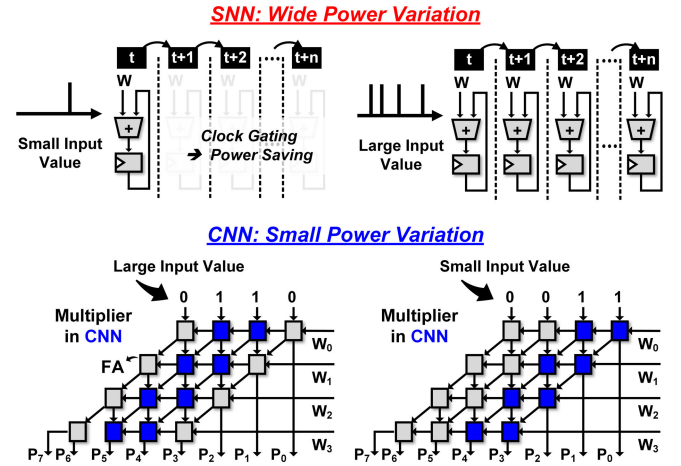


Fig. 4. Power variation of CNN and SNN PE.

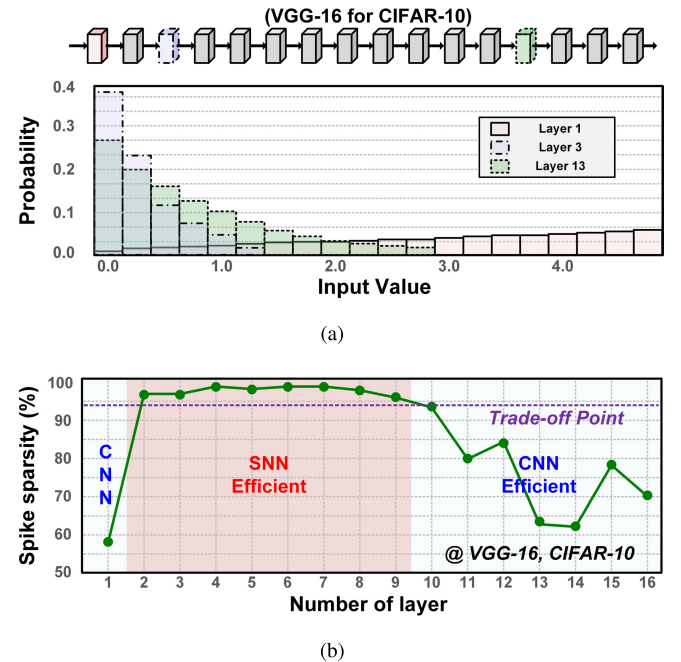


Fig. 5. Complementary characteristics in inference. (a) Input distributions. (b) Spike sparsity and energy-efficient network domain of each layer.

SNN can perform training using only the time difference of pre-neuron's spike and post-neuron's spike without BP and GG, so it is possible to achieve low power consumption. Despite the possibility of achieving low power, the SNN training method has a disadvantage that high accuracy cannot be obtained. CNN with BP can achieve ~40% and ~60% higher accuracy (at CIFAR-10, ImageNet dataset) compared to spike-timing-dependent plasticity (STDP), which is the general training method of SNN. In contrast, SNN can achieve ~70% lower energy consumption for CIFAR-10/ImageNet training compared to the BP of CNN.

Therefore, we propose a complementary deep-neural-network (C-DNN) processor [11] combining CNN and SNN to take advantage of only their strong points in this article. The C-DNN processor has two heterogeneous cores, the CNN core and the SNN core. For low-power inference, workloads

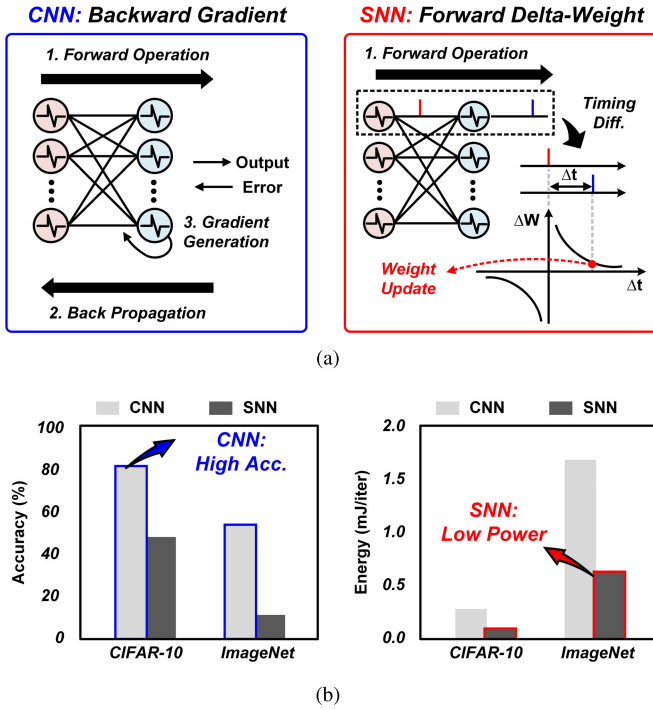


Fig. 6. Complementary characteristics in training. (a) Training method of CNN and SNN. (b) Accuracy and power consumption of each training method.

are divided by layers or within the layer and then allocated to either SNN or CNN, which can consume lower energy. For high energy-efficient training, the forward delta weight of SNN can help CNN to reduce the operations of BP and GG by increasing sparsity. During inference with SNN, the neurons or synapses with large forward delta weight are identified, and only those perform BP and GG of CNN leading to high energy efficiency.

For high energy-efficient C-DNN processing, a C-DNN processor is proposed with five main key features. Compared with previous processors, we increase the energy efficiency by allocating some workload of the CNN into SNN and optimize the SNN inference and forward delta-weight generation. The main key features are given as follows.

- 1) C-DNN inference and training algorithm to reduce energy consumption compared to only CNN or SNN. It can replace multiplier-and-accumulator (MAC) operations by a small number of accumulations. Also, it can perform sparse BP and GG by forward delta-weight-based sparsity generation (FDWSG).
- 2) CNN-SNN workload allocator to enable on-chip workload division and attention module to optimize the performance of CNN-SNN workload division.
- 3) SNN processing element (PE) array with distributed L1 cache to reduce SNN inference power consumption by reusing the weight temporally. It can eliminate the redundant SRAM access.
- 4) Global counter and local delta-weight (LDW) unit to reduce power/area overhead of forward delta-weight generation. It can eliminate the power-consuming

counters that were integrated to record the history of pre- and post-spike.

- 5) Skip index generator to support FDWSG for sparse BP and GG, and sparsity-aware CNN core to increase training energy efficiency by handling the sparsity generated by FDWSG.

The rest of this article is organized as follows. In Section II, the overall architecture of C-DNN processor is described. In Section III, detail algorithm and hardware architecture of C-DNN processor are explained with the proposed key features. Section IV shows the measurement system and results of C-DNN. Then, Section V describes the conclusion of this article.

II. PROPOSED C-DNN PROCESSOR

A. Overall Architecture

Fig. 7 shows the overall architecture of the C-DNN processor with SNN processing unit, CNN processing unit, top reduced instruction set computer (RISC) controller, workload allocator, attention module, global L2 cache (SRAM), skip index generator, 2-D mesh-type network-on-chip (NoC), and neuron link. The SNN processing unit has four SNN clusters, which includes eight SNN cores. The SNN core has 64 spike encoders, 64 linear-feedback shift registers (LFSRs), 64×64 SNN PEs, adder tree, firing logic, global counter, and 64 LDW units. Each spike encoder requests input data to the input memory and converts it to a spike train by using the proposed two-step encoding rule. Also, LFSR is integrated to compensate randomness for generated spike train. It generates a random value at only the first timestep and the value is used for the bias of the spike train. Each SNN PE has an accumulator-based inference unit and LDW accumulator (LDWAC). The inference unit accumulates weight when the pre-spike occurs and generates a partial sum of the neuron's membrane voltage. The LDWAC receives the forward delta weight when both pre-spike and post-spike occur within a specific time step. Each PE has the $4 \text{ b} \times 9$ register file, which is used for the L1 cache, and it enables temporal weight reuse for low-power SNN inference. In addition, each inference unit in SNN PE has a 4-bit weight accumulator, and multiple PEs can be combined to support high bit precision. For example, the SNN core can support 8-bit weight accumulation when two SNN PEs are combined. The adder tree accumulates the partial sums of different PEs and generates a neuron's complete membrane voltage, and the firing logic generates post-spike by comparing the membrane voltage and the threshold. Due to the proposed two-step encoding, we can replace many power-consuming counters with only one global counter. The global counter records the history of pre-spike and shares the coarse time difference between pre-spike and post-spike to different LDW units. Also, each LDW unit computes the fine time difference, converts it to the forward delta weight, and broadcasts it to SNN PEs in the same row.

The CNN processing unit has four CNN clusters, and each cluster has eight CNN cores. CNN core has 8×16 PEs, input memory, input loader, weight memory, and weight loader. The

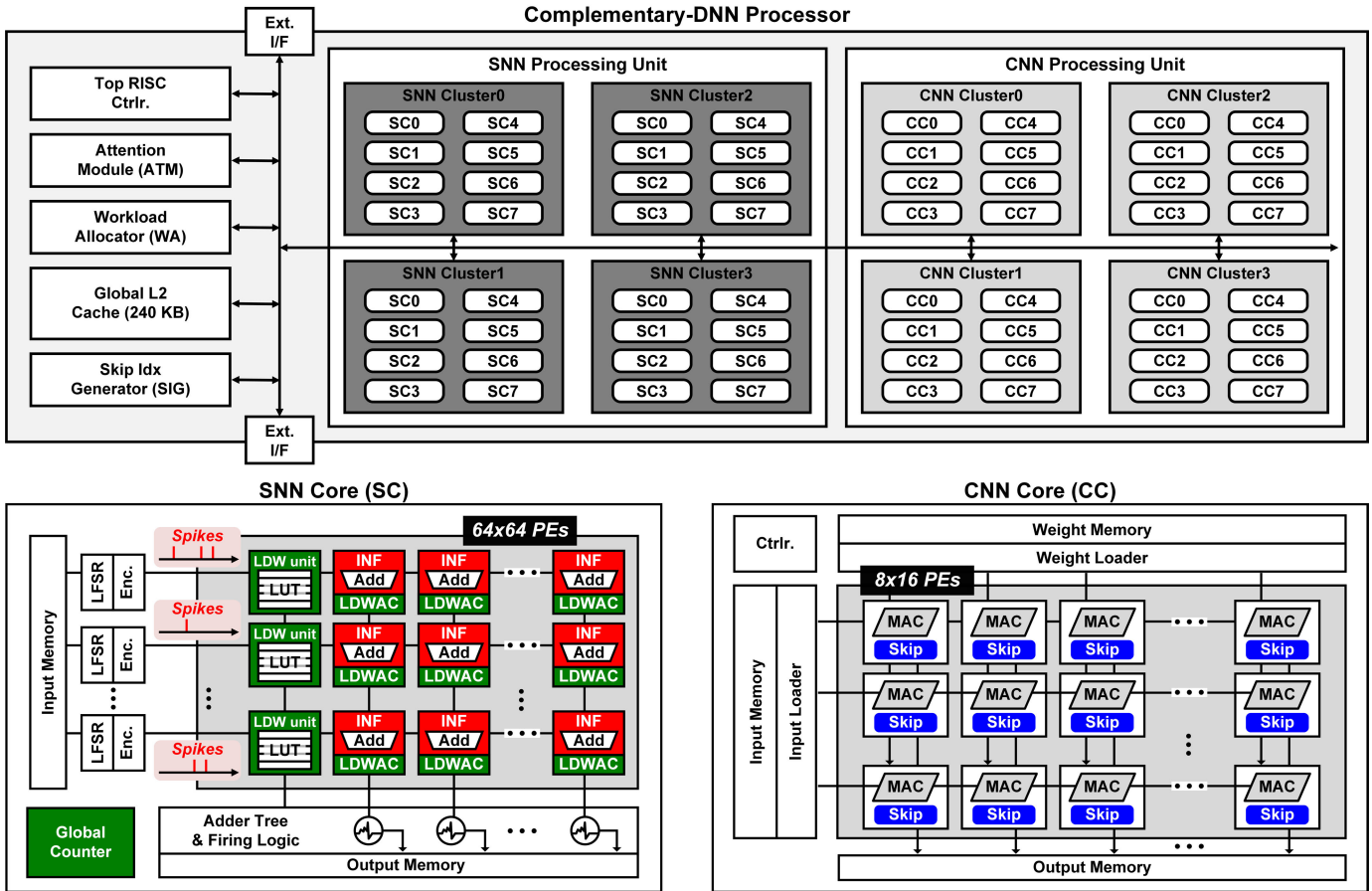


Fig. 7. Overall architecture of C-DNN processor.

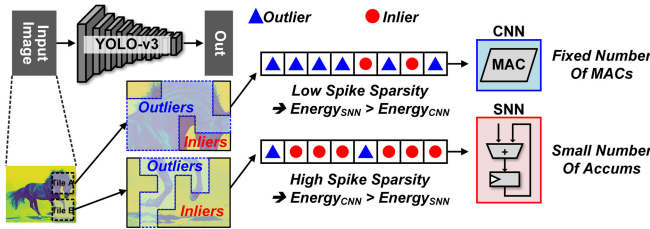


Fig. 8. Motivation of C-DNN inference.

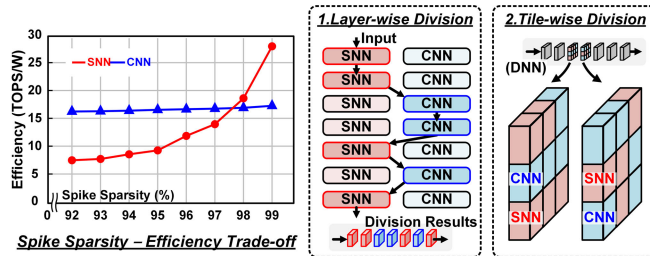


Fig. 9. C-DNN inference.

CNN PE in the CNN core has an MAC and output zero skipping controller. The 8-bit MACs in different PE can also be combined to construct high-precision MAC. The skipping controller receives the sparsity information from the skip index generator and skips the sparse operation, which is generated by FDWSG for high energy-efficient BP and GG. In addition,

the input loader and weight loader enable skipping controller to irregular memory access.

The workload allocator divides the workload of the DNN and allocates to either the CNN or SNN core that consumes low energy. Also, the attention module can reduce the standard deviation of the input distribution to increase the ratio of low-power SNN workload. The skip index generator compares the forward delta weight of the neuron/synapse with the skipping threshold and generates an index that indicates which output neuron can be skipped in BP and GG. The global L2 cache consists of 240-kB SRAM and stores the weights. When each L1 cache requests the weights for a specific workload, the L2 cache transfers them to the L1 cache. In addition, SNN cores and CNN cores are connected with the neuron link [12], which can support aggregation and thresholding between different cores. Therefore, the C-DNN processor has high scalability and it can process not only simple networks such as VGG-16 for CIFAR-10 classification but also complex tasks such as ResNet-50 for ImageNet. In addition, the SNN core of C-DNN has more 64×64 PEs integrated than the CNN core with 8×16 PEs, which is to prevent the increase in latency due to SNN processing. SNNs can have longer latency because they perform computations across multiple time steps using spikes present in the spike train. To address this and match the computational throughput of CNNs and SNNs, we integrate more PEs in the SNN core compared to the CNN core.

B. C-DNN Algorithm

Fig. 8 shows the motivation of C-DNN inference. In the input feature map of DNN, some tiles contain significant parts of objects, and there are many outliers (large values). In this case, a dense spike train is generated through spike encoding, and the CNN core consumes less power than the SNN core. In the case of a tile that does not contain many important parts of an object, spike sparsity is very high due to a lot of small values. Also, in this case, the SNN core consumes less power because it performs only a small number of accumulations. In addition, SNN shows higher energy efficiency than CNN when the spike sparsity is larger than 97.7%, as shown in Fig. 9. Generally, in CNN, one MAC operation corresponds to two operations. In the case of SNN, when the accumulation is performed for the number of time steps, it is equivalent to performing one MAC operation. Therefore, the accumulation for the number of time steps corresponds to two operations. In addition, in SNN, when no spike occurs, accumulation is clock-gated, resulting in a decrease in power consumption. As a result, the frequency of clock gating during the accumulation for the number of time steps varies depending on the frequency of spikes, leading to variations in power consumption during the execution of two operations. Consequently, as spike sparsity increases, the power consumption during the execution of the same operation decreases, leading to an increase in energy efficiency, as shown in the graph in Fig. 9. Therefore, the energy consumption of SNN fluctuates up and down along the layers/tiles depending on the spike sparsity, which changes with each layer/tile, and CNN shows relatively mild variation. If we can divide workloads by layers or within the layer, we can allocate each divided workload to either SNN or CNN, which can consume lower energy. Therefore, we estimate the energy consumption of operations in each divided workload by using spike sparsity and tradeoff information between CNN and SNN, and the divided workload is allocated to more energy-efficient network to perform high-energy-efficient inference. Fig. 10 shows the layer-wise network domain allocation results with layer-wise energy consumption results. In the case of CIFAR-100 classification with ResNet-18, layers 1–3, 5, 8, and 14–16 consume lower energy in CNN domain and layers 4, 6, 7, and 9–13 consume lower energy in the SNN domain. In the case of ImageNet classification with ResNet-18, layers 1, 3, 5, and 8 consume lower energy in the CNN domain and layers 2, 4, 6, 7, and 9–16 consume lower energy in the SNN domain. By operating each layer in optimal network domain, we can reduce energy consumption by 23% for CIFAR-100 classification with ResNet-18 and 31% for ImageNet classification with ResNet-18.

Fig. 11 shows the off-chip SNN conversion and on-chip CNN/SNN switching for C-DNN inference. Before on-chip processing in the C-DNN processor, the highly accurate SNN is generated by the CNN–SNN conversion algorithm in off-chip. It consists of two steps, CNN training with BP and weight sharing. In previous work [9], the accuracy degradation occurs because they use very small time steps (five time steps) for SNN processing. Therefore, we increase the time steps

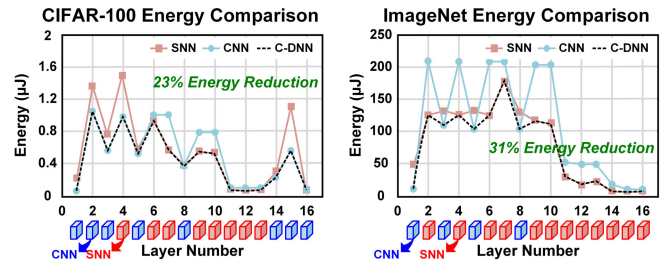


Fig. 10. Results of C-DNN inference.

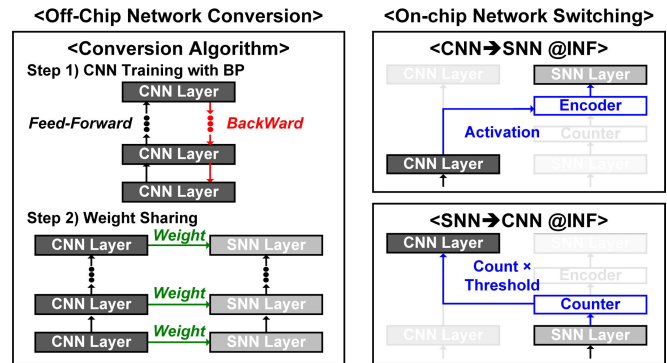


Fig. 11. Off-chip network conversion and on-chip network switching.

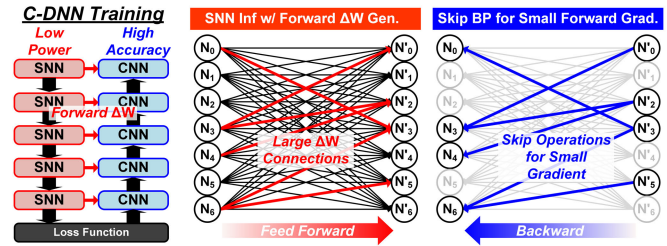


Fig. 12. C-DNN training.

compared to previous work for minimizing accuracy drop due to CNN-to-SNN conversion. With 16 time steps, we can convert VGG16 to highly accurate SNN with an accuracy of 71.6% (0.5% drop) and 69.8% (0.8% drop) for the CIFAR-100 and ImageNet datasets, respectively. After conversion, on-chip processing in the C-DNN processor is performed, and depending on the situation, the pre-trained CNN and the converted SNN may be alternately performed. When switching from CNN to SNN, the output intermediate data of the CNN core are transmitted to the spike encoder in the SNN core by the 2-D NoC. The encoder generates a spike train and the SNN core starts neuron operation. When switching from SNN to CNN, the SNN core counts the number of spikes generated by each neuron and converts the spike train into a scalar value. The converted scalar value is multiplied by the threshold used in the SNN core. This is because the spike is generated whenever the membrane voltage of the neuron exceeds the threshold, so the original scalar value is equivalent to multiplying the spike by the threshold. After that, it is transferred to the CNN core to be used for calculation.

Fig. 12 shows the C-DNN training algorithm to achieve low-power and high-accurate DNN training. For high energy-efficient training, the forward delta weight of SNN can help

CNN reduce the backward operations by increasing sparsity. In the inference stage, only SNN is performed and it computes the forward delta weight of each synapse with low power consumption. Also, these forward delta weights are not used to update the weight due to low accuracy. However, it can be used to predict the magnitude of backward gradient from BP. In the backward stage, before CNN domain BP, it receives the forward delta weight results, which is computed in the inference stage. Also, it predicts the backward paths with small backward gradient, and then, it skips backward operations for negligible backward gradient. After only important backward gradient computation, the weight update is performed with backward gradient for high accuracy. As a result, forward delta weight of SNN can generate sparsity in BP, and we can increase the training energy efficiency by skipping high-cost backward operations.

Fig. 13 shows the detail of FDWSG in C-DNN training. During the SNN inference, the forward delta weight is calculated for each synapse by the forward delta weight logic in the SNN PE. For sparse BP, the average of the forward delta weights of pre-synapses connected to the neuron is computed. If the average is smaller than the skipping threshold, the C-DNN processor predicts that BP for the neuron is not required because the gradients of the connected pre-neurons will be small. For sparse GG, the forward delta weight is compared with the skipping threshold for each synapse of the neuron in which the BP has been processed, and operations for the negligible gradients are predicted and skipped for energy reduction. In summary, only SNN forward pass is performed for forward delta-weight sparsity generation in inference mode, and only CNN backward pass is performed with high sparsity for weight update.

As a result, 58% and 79% of computations of BP and GG are skipped for ResNet-18 training with the CIFAR-10 dataset, as shown in Fig. 14. Also, 44% and 64% of BP and GG are skipped for ResNet-18 training with the CIFAR-100 dataset. In the case of ImageNet training, 31% and 43% operations are skipped for ResNet-18.

C. CNN-SNN Workload Allocator and Attention Module

Fig. 15 shows the CNN-SNN workload allocator. The number of spikes is proportional to the magnitude of data. After the operation is performed in the previous core, the intermediate data for a specific tile are moved to the output memory. At this time, the number of spikes in the tile is obtained by accumulating data in each core, and it is passed to the workload allocator. Then, workload is allocated to either the CNN or SNN core, which shows lower energy consumption. For this, we first measured the power consumption in CNN and SNN cores based on spike sparsity. This information is transferred to the workload allocator by instructions. In each core, when the computation for a specific tile is completed and intermediate data are generated, the spike sparsity of that data is calculated. This spike sparsity information is then transmitted to the workload allocator. The workload allocator predicts the power consumption in each

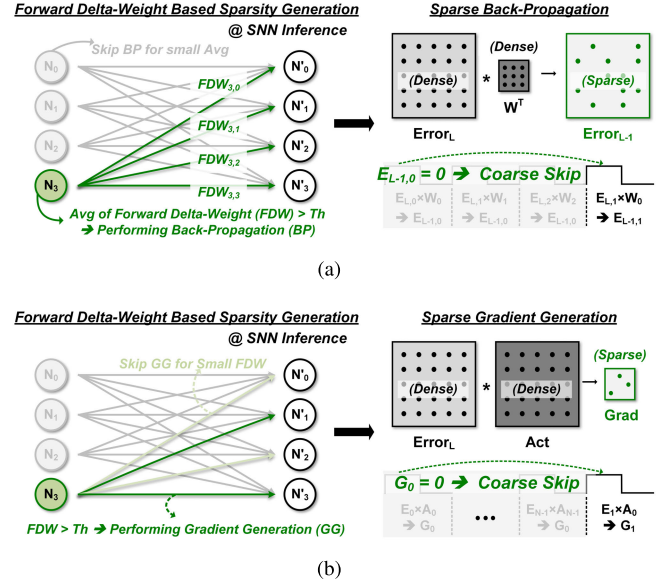


Fig. 13. FDWSG. (a) For sparse BP. (b) For sparse GG.

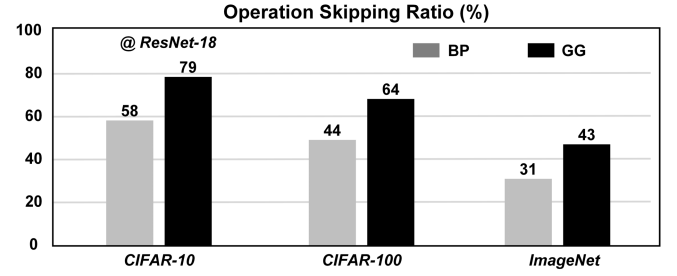


Fig. 14. Performance of FDWSG.

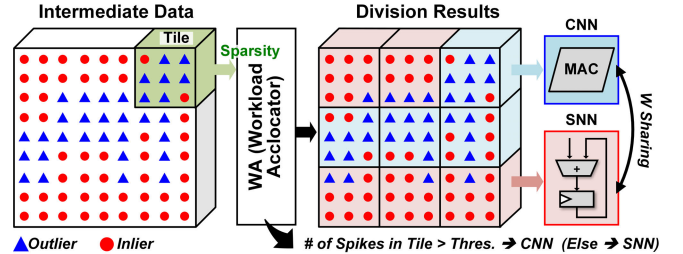


Fig. 15. CNN-SNN workload allocator.

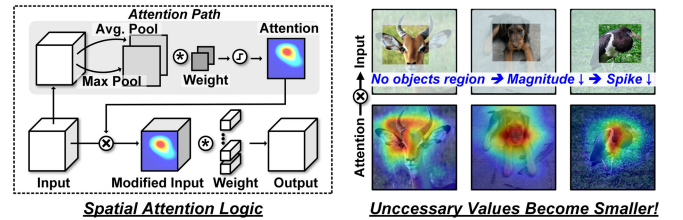


Fig. 16. Attention logic to optimize CNN-SNN workload division.

core based on the premeasured power consumption results and the pre-received spike sparsity. Subsequently, the workload allocator assigns the computation of the next layer to the core that consumes less power than the two types of cores. ResNet-18 with ImageNet shows an average of 94.5% spike

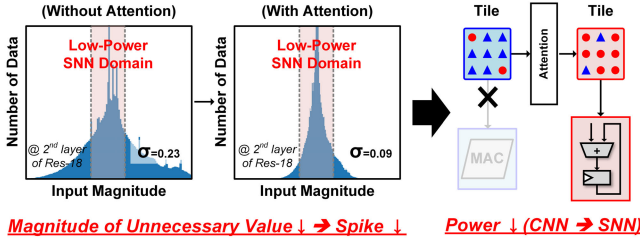


Fig. 17. Effect of attention logic.

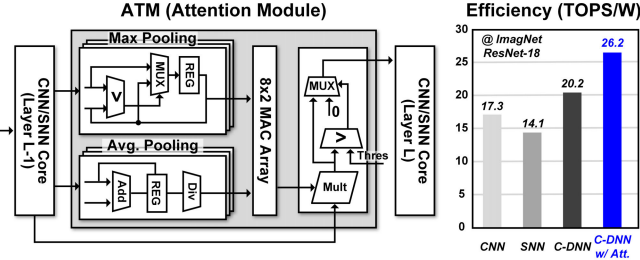


Fig. 18. Attention module and performance.

sparsity, and energy efficiency increases by 43.3% compared to SNN and 16.7% compared to CNN.

In addition, we suggest a C-DNN optimization that incorporates attention to improve energy efficiency, as shown in Fig. 16. The attention map is obtained by performing channel-wise pooling on the input feature map, followed by a small convolution using pre-trained weights for attention. The resulting map is then multiplied with the input feature map to reduce the values of unimportant regions, leading to an increase in spike sparsity and low-power SNN core workload without affecting accuracy. Moreover, in ImageNet, objects are sparsely distributed across the 2-D image plane, allowing the attention module to reduce the input distribution. For instance, in the second layer of ResNet-18, the standard deviation of the input feature map decreases from 0.23 to 0.09, as shown in Fig. 17, resulting in increased spike sparsity and more workloads allocated to low-power SNN without sacrificing accuracy. Fig. 18 shows the attention module, which is composed of the max pooling unit, average pooling unit, 8×2 MAC array, and multiplier. The results of previous layer are transferred to the attention module, and it is multiplied to the attention map. After attention, the modified input feature map is transferred to the next core for the next layer operation. The C-DNN processor with attention module operation achieves 97.9% spike sparsity for ResNet-18 with ImageNet and 26.2 TOPS/W, which is 85.8% higher than SNN and 51.4% higher than CNN.

In addition, we referred to the C-BAM [13] to train the network with attention module. We add spatial attention paths and channel attention paths, perform inference to calculate the loss, and then backpropagate the loss to update the weights of MLPs or convolution layers existing in each attention path, thereby generating pretrained weights. As described in [13], attention techniques increase the accuracy by emphasizing important parts. In [13], ResNet50 without attention exhibited

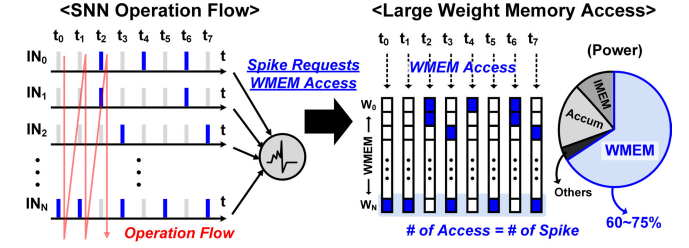


Fig. 19. Challenge for low-power SNN inference.

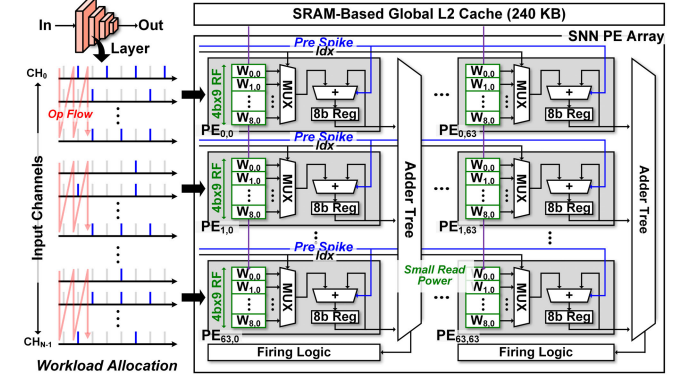


Fig. 20. SNN PE array with distributed L1 cache.

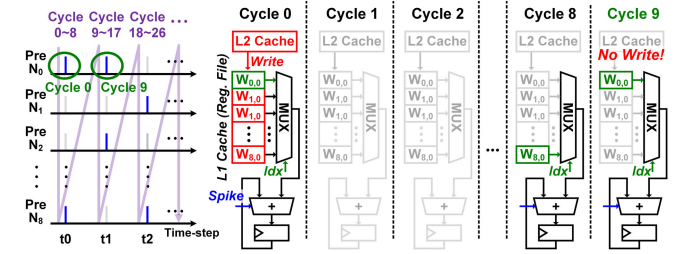


Fig. 21. Operation of SNN PE array with distributed L1 cache.

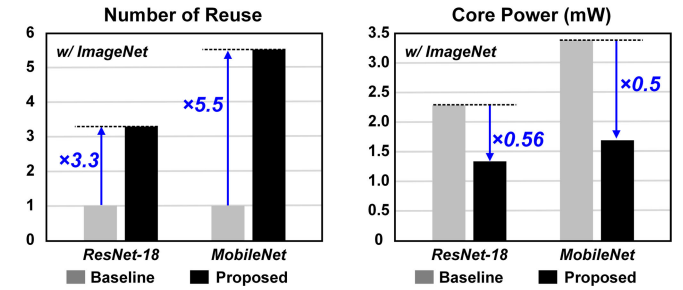


Fig. 22. Performance of SNN PE array with distributed L1 cache.

a top-1 error rate of 24.56%. However, when attention was applied to ResNet50, it showed a reduced top-1 error rate of 22.8%. Similarly, in the case of C-DNN, when attention is applied, it shows an accuracy improvement of 1.89%.

D. SNN PE Array With Distributed L1 Cache

Fig. 19 shows the challenges for complementary DNN processor. In the case of SNN, it performs integrate-and-fire function during time window. In this process, SNN performs weight integration in the channel direction first, and then,

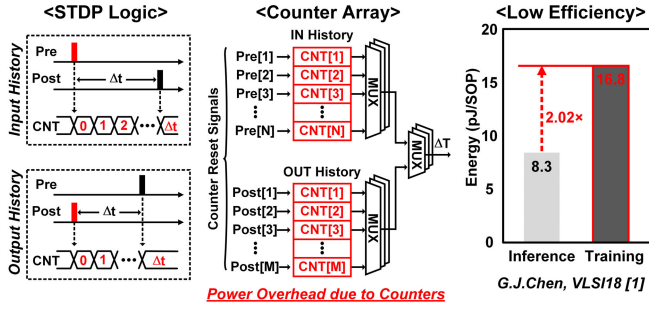


Fig. 23. Challenge of low-power forward delta-weight generation.

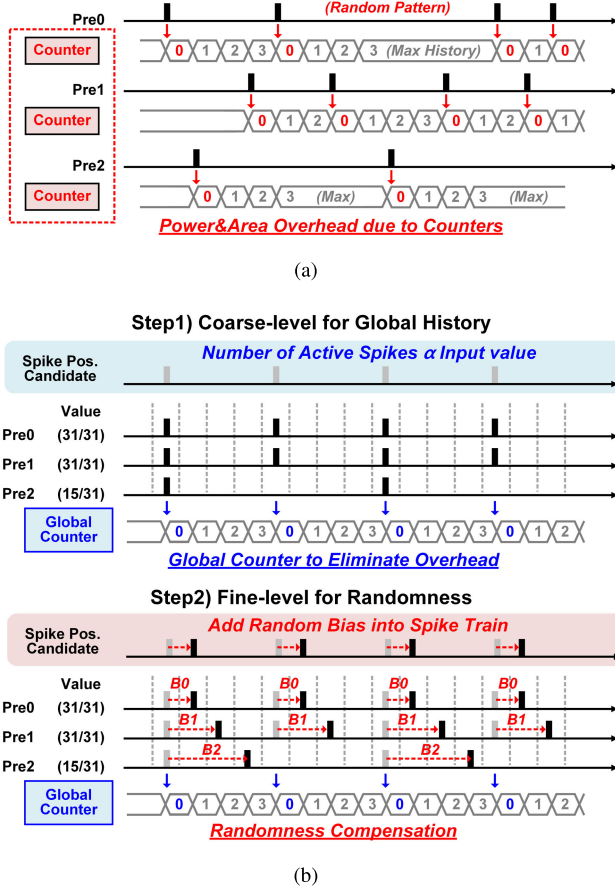


Fig. 24. (a) Previous spike encoding. (b) Proposed two-step spike encoding.

it performs a firing operation. After that, it starts the integrate-and-fire operation of next time step. If the same pre-neuron fires pre-spike at the next time step, the same weights should be fetched again. Thus, the same weight SRAM memory address is accessed multiple times, resulting in $\sim 70\%$ power consumption of total inference power. To solve this challenge, we propose SNN PE array architecture, which can reuse fetched weight data, as shown in Fig. 20. Each SNN PE has inference logic with an accumulator and L1 cache (9×4 b register file) for temporal weight reuse. The spike encoder converts the input data of the pre-neuron to the spike train, and the generated pre-spike is transferred to the SNN PEs in the same row by broadcasting.

Fig. 21 shows the operations of SNN PE with L1 cache. Before the SNN operation, the weights for the nine

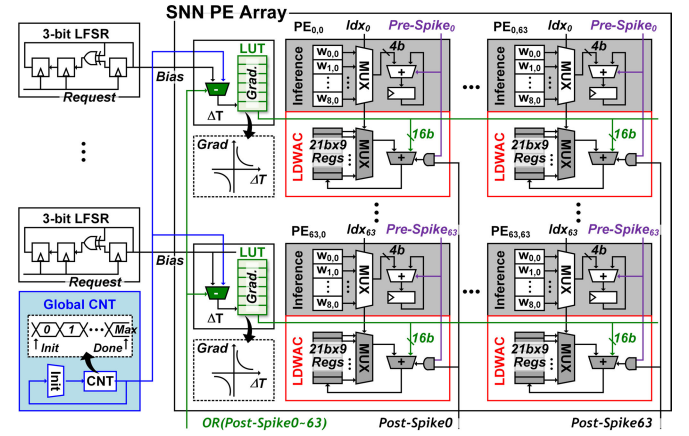


Fig. 25. Forward delta-weight logic with global counter and LDW unit.

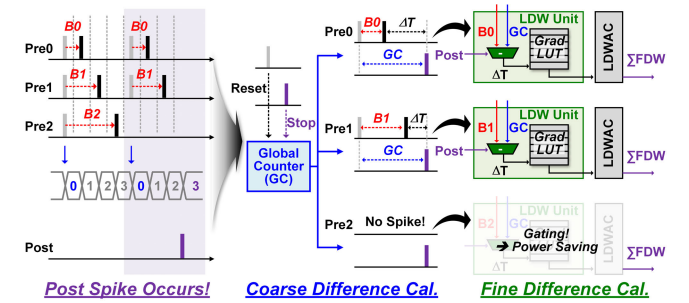


Fig. 26. Operation of forward delta-weight logic.

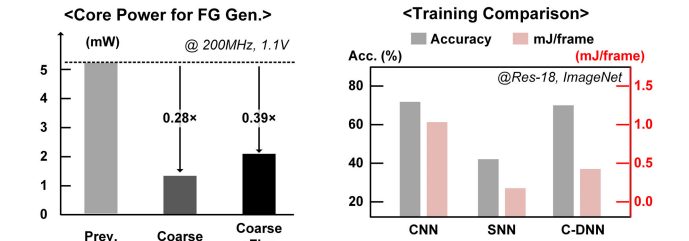


Fig. 27. Performance of forward delta-weight logic.

pre-neurons are fetched from the global L2 cache (SRAM) to each L1 cache. At cycle 0, the spike of pre-neuron 0 is transferred to the SNN PE. Thus, the weight corresponding to the pre-neuron 0 is selected by the index from the spike encoder, and the weight from the L1 cache is accumulated with a partial sum. At cycles 1 and 2, the SNN PE does not receive the spike from pre-neurons 1 and 2. In this case, the weight read operation and accumulation are not performed by clock gating for power reduction. At cycle 8, the spike operation for pre-neuron 8 is performed in SNN PE. During eight cycles, all spikes for all pre-neurons connected to the post-neuron are transferred, and the weight accumulations in the first time step (t_0) are completed after cycle 8. After that, the adder tree generates membrane voltage by column-wise addition between PEs, and a firing operation is performed. After one time-step SNN operation, at cycle 9, the pre-spikes of the next time step are transferred to PEs, and the weights in the L1 cache are reused instead of SRAM access. This

operation is repeated and the weights in the L1 cache can be reused until the last time step.

In addition, we aimed to design a reconfigurable architecture with hierarchical cache to handle DNNs of any structure. Each layer in the DNN typically has a channel size that is a multiple of 64, and the most commonly used weight size is 3×3 . Therefore, the most common operation unit in DNN computations is the operation between a weight of size $3 \times 3 \times 64$ and an input. Each PE column in a single core consists of 64 PEs, and to accommodate all the $3 \times 3 \times 64$ weights in a column, we designed each PE to hold nine weights. In other words, in a single core, a $3 \times 3 \times 64$ kernel is stored, and this kernel performs convolution operations in the SNN domain with the input feature map of a specific layer. Once the kernel completes all the operations by sweeping through the input feature map, it is no longer reused. At this point, the L1 cache requests a new kernel from the L2 cache to update its contents. The exact number of cycles may vary depending on the width and height of the input feature map. Fig. 22 shows the performance of SNN PE array with distributed L1 cache. By eliminating the repeated SRAM accesses, the reusability increases by $3.3\times$ and $5.5\times$ for ImageNet classification with ResNet-18 and MobileNet, respectively. Moreover, the power consumption of the SNN core decreases by 42.2% and 49.1% for ImageNet classification with ResNet-18 and MobileNet, respectively.

E. Global Counter and LDW Units in SNN Core

Fig. 23 shows challenges for C-DNN training. For low-power C-DNN training, we must compute forward delta weight with low power consumption. However, previous SNN ASICs with STDP-based training [1], [2] could not achieve low-cost forward delta-weight generation. They integrated a lot of counters to record the history of pre-spike and post-spike. After pre-spike occurs, the counter increased until post-spike occurs to compute the time difference. Even if a post-spike does not occur, the counter should be operated because it was not possible to predict if a post-spike would occur later. Therefore, many counters should perform redundant operations and occur large power and area overhead. As a result, the SNN training mode with forward delta-weight generation in the previous ASIC achieved $2\times$ power consumption compared to the inference mode.

To solve this challenge, we propose two-step spike encoding, as shown in Fig. 24. Previous spike encoding generates random pattern spikes, so each neuron should have individual counter to record the history. However, in Step 1 of the coarse encoding, the spike encoder can generate spike only at the specific times that are represented by a gray line, and the spike occurrence probability is proportional to the input magnitude. Therefore, multiple pre-neurons generate spikes at the same time, and the neurons can share the same global counter to record the history of pre-spike. In Step 2 of fine encoding, the spike encoder compensates randomness of previous rate encoding by adding a random bias

(B0, B1, ..., B63) generated by LFSR into each spike train. The conventional Poisson rate coding involves multiple pre-spikes occurring at random time steps. As a result, the weights of each synapse are added at different time steps, and the time difference between the post-spike occurrence and the arrival of the pre-spikes at each synapse varies. Therefore, the conventional method is suitable for determining which synapses are important. However, if only the proposed Step-1 encoding is performed, multiple pre-spikes occurred at the same time step, causing the weights of multiple synapses to be added simultaneously. In this case, it becomes difficult to determine which weights present in synapses significantly influenced the spike occurrence, leading to a decrease in the accuracy of predicting the size of backward gradients for C-DNN training. To address this, we apply a random bias to each pre-spike train, ensuring that the weights of each synapse are added at different time points. Without using random bias, during the training of ResNet-18 to achieve an accuracy of 71.2% for ImageNet classification, the operations that can be omitted in BP and GG through FDWSG are only 3% and 5%, respectively. In contrast, when using random bias, as described in Section II-B, a much larger percentage of operations can be skipped, amounting to 31% and 43%. The LFSR operates exactly once when a specific input value enters the core to coincide with the start of Step-1 encoding, generating a random value. The spikes produced by Step-1 encoding are delayed by the generated random cycle and then passed to the PEs, where Step-2 encoding is performed. Once the operation for a specified time window is completed, the next input value enters for the next operation, and at this point, the LFSR operates once again, together with Step-1 encoding. Unlike the training mode described so far, in the case of the inference mode, LFSR does not operate and the bias is fixed because the inference mode does not require FDWSG.

After Step-2 encoding, when a post-spike occurs, the spike time difference can be easily obtained by calculating the difference between the global counter value and the bias determined for each neuron. In the previous method, the counters are always operated to record the pre-spike history regardless of whether the post-spike occurred or not. However, in the proposed method, only one global counter always operates, and only when a post-spike occurs, the difference between the global counter and the bias is calculated through a low-bit subtractor. Therefore, we can reduce the 61%–72% power consumption by eliminating redundant counter operations. In addition, a new encoding method is adopted instead of Poisson spike encoding, but the accuracy is not significantly compromised. When conducting CNN-to-SNN conversion, we generated an SNN model by applying the proposed spike encoding. By using the proposed spike encoding and finding the threshold value that demonstrates high accuracy for each neuron, we achieved accuracy that is nearly comparable to the conversion method using conventional Poisson spike encoding. The conventional Poisson encoding shows an accuracy of 75.32% in the ImageNet classification, while the proposed encoding demonstrates an accuracy of 75.21%.

Fig. 25 shows the forward delta-weight logic in the SNN PE array for the proposed two-step encoding. The logic consists of the global counter, 64 LDW units, 64 LFSRs, and 64×64 LDWACs. Also, each column of SNN PE array computes the integrate-and-fire operation of each post-neuron, and each PE in column performs forward delta-weight generation of each synapse between post-neuron and each pre-neuron simultaneously. Fig. 26 shows forward delta-weight logic operation. A reset signal is given to the global counter at each possible spike occurrence time step (gray line) defined in Step 1 of the proposed spike encoding, and the value of the counter is initialized to 0 because the duration of spike occurrence time step is equal to maximum history count. In the training method of SNN, if the time difference between pre-spike and post-spike exceeds max history, it is judged that there is no correlation between the two spikes, and the forward delta weight is not need to be obtained, so the history can be reset. After that, the counter continues to increase until a post-spike occurs, and the increased counter value is broadcast to 64 LDW units when one of the post-neuron fires the post-spike. To implement this logic in the SNN PE array, the OR operation result between post-spikes triggers the LDW unit computation. The time difference between pre-spike and post-spike is obtained for each pre-neuron's unbiased spike train by a global counter, and each LDW unit obtains the complete time difference by subtracting the global counter value from the bias applied to each pre-neuron. Then, the spike time difference is converted to forward delta weight by the lookup table, and delta weight is broadcast to LDWACs in the same row. During this operation, if pre-spike does not occur like pre-neuron 2, the LDW unit does not need to compute the spike time difference, and it reduces power consumption by clock gating. To implement this, each PE receives the corresponding pre-spike and post-spike, and then, it performs AND operation to trigger the operation of the LDWAC. Through this, it is possible to omit the calculation of the forward delta weight for a synapse in which a pre-spike occurred but no post-spike occurred, and at the same time, the calculation for a synapse in which a post-spike occurred but no pre-spike occurred can be omitted. The calculated forward delta weight is sequentially accumulated in LDWAC to generate a complete forward delta weight. To sum it up, energy consumption is reduced by replacing numerous redundant counters with a single global counter and a low-bit subtractor with event-driven operation. Fig. 27 shows the performance of forward delta-weight logic. This feature can reduce the SNN core power consumption for both inference and forward delta-weight generation by 61%. Moreover, FDWSG increases the training energy efficiency by 61.6% and 28.7% for CIFAR-10 and ImageNet, respectively. Also, the C-DNN training shows a 0.72% accuracy drop after entire epochs compared to only CNN training, which shows 71.9% top-1 accuracy for ImageNet classification with ResNet-18. However, it shows 28.7% higher accuracy compared to SNN training, which shows only 42.5% accuracy for ImageNet classification. In summary, at the negligible sacrifice of accuracy, C-DNN training achieves a very large energy efficiency increase.

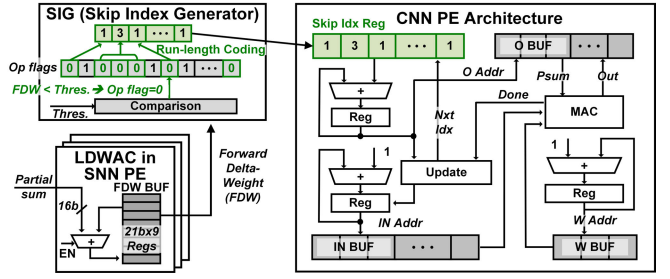


Fig. 28. Skip index generator and sparse CNN PE architecture.

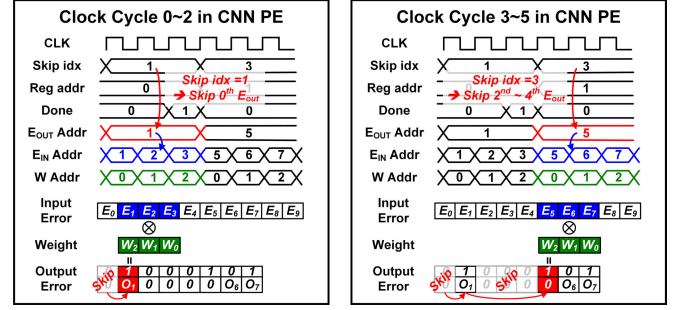


Fig. 29. Operation waveforms of sparse CNN core.

F. Skip Index Generator and Sparse CNN Core

The skip index generator is integrated to predict weights that do not need to be updated, as shown in Fig. 28. It has a comparator and zero-length encoder and receives forward delta weight from the LDWAC in the SNN PE. The skip index generator compares the forward delta weight and skipping threshold and generates operation flags (the “0” flag means sparse operation). Finally, it performs run-length encoding for operation flags and transfers the length of sparse operations to the skip index register in the CNN PE. In detail, the skip index generator averages forward delta weights of pre-synapses connected to the neuron, and if the average is smaller than the threshold, the generator orders CNN PE to skip BP for the neuron. Also, it compares the skipping threshold with forward delta weight for each synapse of the neuron to determine whether GG can be skipped.

The sparse CNN PE consists of the 8-bit fixed point MAC and address controller for input, weight, and output. To generate one output in address #0 of the output buffer by using row-stationary data reuse [14], three input data (addresses #0, #1, and #2 in the input buffer) and three weight data (addresses #0, #1, and #2 in the weight buffer) are required (at 3×3 kernel convolution case). The address controller initializes all addresses to #0 and increases the input and weight address to generate output. Then, the convolution is performed in the MAC and the result is stored to address #0, and a done signal is transferred to update logic to generate the next addresses. When the done signal occurs in the MAC, the address controller increases the output address by 1. In addition, there is additional logic to skip the computations for output zeros. The update logic in the controller reads the “0” flag length in the skip index register that indicates the number of output addresses for the CNN PE to skip. The address controller additionally increments the

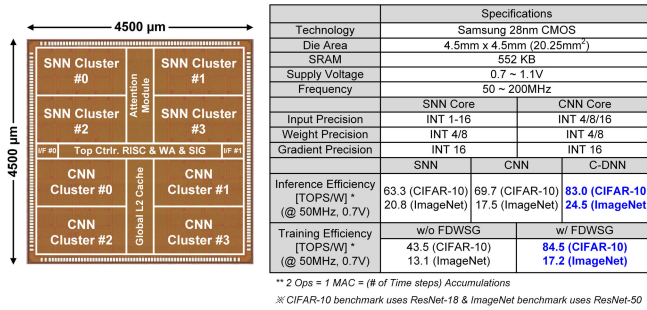


Fig. 30. Chip photograph and performance summary.

output address by the “0” flag length to generate the address of the next non-sparse output. If there is no sparsity (“0” flag length = 0) and the previous output address is #0, then the next output address becomes address #1, and new three inputs (addresses #1, #2, and #3) and weights (addresses #0, #1, and #2) are fetched for output computation. In contrast, if the next output is sparse (“0” flag length = 1) and the previous output address is #0, the next output address becomes address #2, and new three inputs (addresses #2, #3, and #4) and weights (addresses #0, #1, and #2) are fetched for output computation. Therefore, input and weight fetching for the output of address #1 are skipped and the output buffer does not receive the corresponding MAC result.

Fig. 29 shows the example operation waveforms of the CNN PE in the sparse CNN core. At cycle 0, the first skip index is 1, so the computations for the 0th output are skipped and the inputs from addresses #1 to #3 and weights from addresses #0 to #2 are loaded in order to compute the 1st output data. During three cycles, the MAC computes the 1st output error, and then, it generates a done signal and the update logic prepares to fetch the next data to compute the next non-zero output. At cycle 4, the next skip index is fetched from the skip index register. The second skip index is 3, so the computations for the second, third, and fourth output data are skipped and the fifth output computation is started in CNN PE. Thus, the inputs from addresses #5 to #7 and weights from addresses #0 to #2 are loaded in order to compute the 5th output data. By the FDWSG and sparse CNN core, 58% and 79% of operations in BP and GG are reduced for ResNet-18 training with the CIFAR-10. In addition, 31% and 43% of BP and GG are skipped for ResNet-18 training with the ImageNet.

III. IMPLEMENTATION RESULTS

Fig. 30 shows the chip photograph of the C-DNN processor, which is fabricated in Samsung 28-nm CMOS technology, and its area is 20.25 mm². Its supply voltage is 0.7–1.1 V and the operating frequency is 50–200 MHz. It integrates the SNN core and CNN core and supports SNN single mode, CNN single mode, and C-DNN mode. The SNN core supports INT 1–16 bit precision for input and INT 4/8 bit for weight and INT 16 for the delta weight. The CNN core supports INT 4/8/16 bit precision for input and INT 4/8 bit for weight and INT 16 bit for the gradient. In the SNN single mode, the C-DNN processor consumes 42 mW at 50 MHz

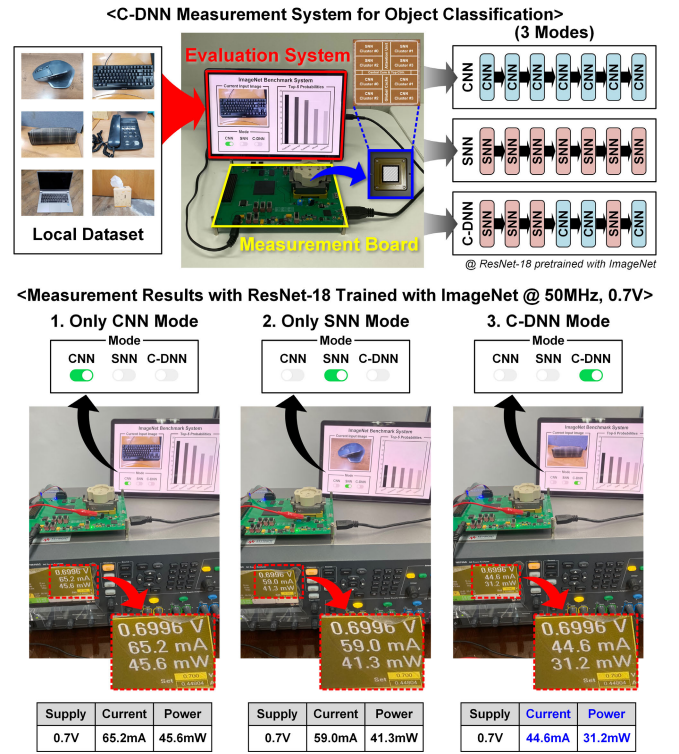


Fig. 31. Measurement system of C-DNN processor.

and 0.7 V and 384.4 mW at 200 MHz. In addition, the C-DNN mode consumes 32.3 mW at 50 MHz with 0.7 V, and 294.1 mW at 200 MHz with 1.1 V, resulting in a 23.5%–33.1% reduction compared to SNN/CNN. In this case, the throughput of all modes is the same for fair comparison. Also, the peak throughput of each mode is given as follows. The CNN-single mode shows 1.64 TOPS, the SNN-single mode shows 3.28 TOPS at 16 time steps, and the C-DNN mode shows 4.92 TOPS. In the SNN single mode, only SNN cores operate, and in the CNN single mode, only CNN cores operate. Also, the workload allocator does not work in both modes, it only works in C-DNN mode. In addition, due to the structural differences in the PE arrays, with more PEs integrated into SNN cores compared to CNN cores, the SNN mode exhibits higher throughput. The reason why more PEs are integrated into the SNN core is that the workload allocated to SNN by C-DNN workload allocation is higher than that allocated to CNN. There are two reasons for workload imbalance. First, the activations of DNN follow a normal distribution, which means that there are more small values than large values. Second, attention further increases the number of small values. Due to these two reasons, there is a higher distribution of small values that generate fewer spikes, resulting in a higher workload for SNN compared to CNN. Therefore, SNN PEs are more integrated than CNN PEs.

In the real operation case with C-DNN mode, the throughput can be degraded compared to peak performance because of the C-DNN workload allocation. If we want to operate each tile only in the optimal domain, an imbalance between the CNN workload and SNN workload can occur. This can

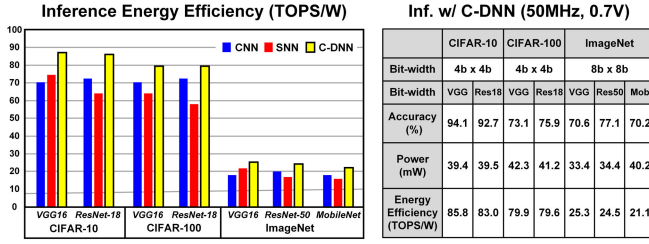


Fig. 32. Energy efficiency benchmark test.

result in certain cores not receiving assigned computations, leading to decreased utilization. However, in scenarios where both energy efficiency and latency are crucial, an optimal approach in C-DNN is to allocate the workload to cores in different domains when the optimal domains are all busy. In this way, we can achieve higher utilization, and both energy and latency can be optimized. Moreover, it achieves 56% energy reduction for ImageNet training (entire epochs) by the FDWSG. The C-DNN processor without FDWSG consumes an energy efficiency of 43.5 and 13.1 TOPS/W for CIFAR-10 and ImageNet training, respectively. By using FDWSG, it can achieve an energy efficiency of 84.5 and 17.2 TOPS/W for CIFAR-10 and ImageNet training, respectively.

Fig. 31 shows the measurement system for the C-DNN processor. We measure the C-DNN processor at three modes (CNN single mode, SNN single mode, and C-DNN mode) and use ResNet-18, which is pre-trained with the ImageNet dataset to classify the local dataset. The C-DNN processor consumes 45.6 and 41.3 mW for CNN single mode and SNN single mode, respectively. In the C-DNN mode, the proposed processor consumes only 31.2 mW for image classification at the same throughput condition, in resulting at least 24.5% power reduction compared to single mode.

Fig. 32 shows the results of the inference benchmark test with the C-DNN processor. The C-DNN can support the CNN single mode, SNN single mode, and C-DNN mode. In the single mode, only one type of core among the CNN core and SNN core operates and the other core becomes an idle state. In the C-DNN mode, two heterogeneous cores perform inference simultaneously by the CNN-SNN workload division. For the CIFAR-10 classification with VGG-16 at 0.7 V and 50-MHz condition, the CNN single mode and the SNN single mode achieve the energy efficiency of 69.83 and 74.45 TOPS/W, respectively. The C-DNN mode achieves an energy efficiency of 85.83 TOPS/W at the same condition, which is 23%/15% higher than the CNN/SNN single mode. Also, the C-DNN mode achieves 19%/31% improvement for ResNet-18.

For the ImageNet classification with ResNet-50, the C-DNN mode can achieve higher performance improvement by the attention module. In the case of an input image in the ImageNet dataset, the area of that object located in the entire image is small. The attention module reduces the spike ratio of the input feature map. Therefore, attention leads to an increased workload of the low-power SNN core and reduced power consumption. As a result, the C-DNN mode increases the energy efficiency by 45%/11% (VGG-16),

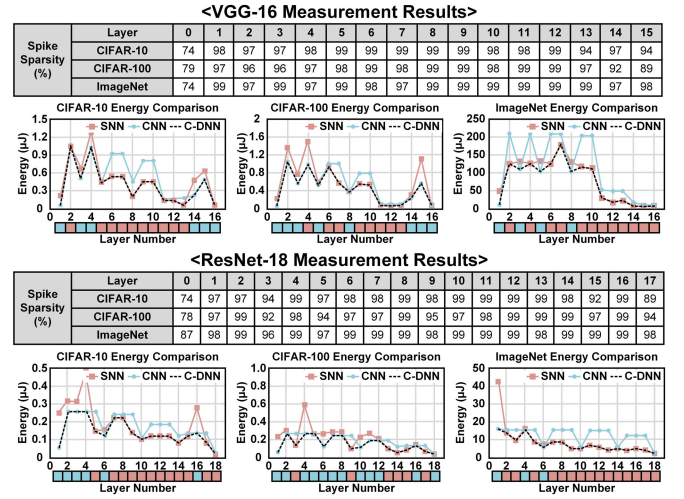


Fig. 33. Spike sparsity of each layer and energy consumption of each mode.

TABLE I
COMPARISON TABLE OF C-DNN PROCESSOR

	ISSCC22 [15]	ISSCC22 [16]	JSSC21 [17]	ISSCC21 [18]	ISSCC21 [19]	This Work
Tech. (nm)	40	28	28	28	40	28
Supply Voltage (V)	0.8-1.1	0.5	0.58-1.04	0.6-0.9	0.75-1.1	0.7-1.1
Frequency (MHz)	147-614	13-115	250	100-470	180	50-200
Die Area (mm ²)	9	0.86	12.96	1.9	7.29	20.25
Target Domain	CNN	SNN	CNN	CNN	CNN	CNN/SNN
Training	X	O	O	X	O	O
Activation Precision	INT 8	INT16	INT 4/8/12/16	INT 8	FP8-SEB	INT 4/8/12/16 (CNN) INT 1-16 (SNN)
Weight Precision	INT 1	INT 1-16	INT 4/8/12/16	INT 8	FP8-SEB	INT 4/8
Inference Energy Efficiency	16.0 TOPS/W (ImageNet)	5.3 pJ/SOP (DVS Gesture)	15.2 TOPS/W (ImageNet)	12.6 TOPS/W (ImageNet)	N/A	24.5 TOPS/W ** 1.1 pJ/SOP *** (ImageNet, ResNet-50)
Training Energy Efficiency	N/A	9.4 pJ/SOP * (DVS Gesture)	10.7 TOPS/W (ImageNet)	N/A	1.6 TFLOPS/W (ImageNet)	17.2 TOPS/W ** 2.8 pJ/SOP *** (ImageNet, ResNet-50)
Skipped Operation Ratio (BP / GG)	0% / 0%	0% / 0%	N/A	N/A	0% / 0%	31% / 43% (ResNet-18)
Training Accuracy	70.09 (ResNet-50)	87.3 (DVS Gesture)	N/A	76.92 (ResNet-50)	69.0 (ResNet-18)	77.1** (ResNet-50) 75.2 (ResNet-50) 71.2 (ResNet-18)

* Scaled by Power Measurement Result ** C-DNN Mode with Attention *** J/SOP = (Energy/Frame) / (# of Spikes)

50%/31% (ResNet-18), and 21%/29% (MobileNet) compared to the CNN/SNN single mode at the same condition. In addition, the bit precision of input and weight, accuracy, power consumption, and energy efficiency for each benchmark are described in the right of Fig. 32.

The table in Fig. 33 shows the detail CNN/SNN workload division results for different networks such as VGG-16 and ResNet-18 and different datasets such as CIFAR-10, CIFAR-100, and ImageNet. In particular, it explains the layer-wise spike sparsity and energy consumption of three different modes (CNN single mode, SNN single mode, and C-DNN mode). As the layer is changed, the energy-efficient network domain is changed and the C-DNN processor can operate in the optimal domain. For CIFAR-10 classification, the C-DNN mode can increase the energy efficiency by 15%–31% compared to the CNN single mode and SNN single mode. Moreover, the C-DNN increases the energy efficiency by 15%–38% for CIFAR-100 and 11%–50% for ImageNet.

Table I shows the comparison table of the C-DNN processor. The previous works [15], [16], [17], [18], [19] can process only one type of network, and they cannot achieve high energy efficiency because there is a magnitude-energy tradeoff between CNN and SNN. The previous state-of-the-art inference processor [15] for CNN achieves the energy

efficiency of 16.0 TOPS/W for ImageNet classification with ResNet-50. However, it has a critical accuracy degradation issue due to low weight precision (1 bit). Moreover, it cannot reduce the power consumption for small-magnitude inputs because it can support only fixed 8-bit activation. As a result, the C-DNN processor can achieve 53.1% higher energy efficiency compared to the previous state-of-the-art inference processor [15]. In addition, the previous state-of-the-art training processor [17] for CNN achieves an energy efficiency of 10.7 TOPS/W by using bit-slice processing. It can reduce power consumption by reducing the bit precision for the MAC operation. However, the multiplier of CNN still consumes larger power than the adder of SNN at the high spike sparsity condition. Also, it cannot utilize the chance to skip the unnecessary BP and GG for weight, which does not need to be updated. Due to the FDWSG, the C-DNN processor can achieve the energy efficiency of 17.2 TOPS/W for ImageNet training, which is 60.7% better than the previous state-of-the-art CNN training processor [17]. In the case of comparison with the previous SNN processor [16], it achieves energy per spike operation of 5.3 pJ/SOP at the SNN inference and 9.4 pJ/SOP at the SNN training. However, it can process only simple tasks such as DVS Gesture classification and keyword spotting. Not only can the C-DNN processor handle complex datasets like ImageNet, but it also reduces energy per spike operation (pJ/SOP) by 79% in inference by 70% in training. Also, the C-DNN processor shows 77.1% training accuracy with attention module operation for ImageNet classification (ResNet-50). Without attention, C-DNN can achieve 75.21% training accuracy for ImageNet classification with ResNet-50. The baseline accuracy with full precision training is 75.93% (without attention), and this means that the C-DNN mode achieves high energy efficiency with negligible accuracy degradation. Moreover, C-DNN can achieve 71.2% training accuracy (0.8% accuracy drop) for ImageNet classification with ResNet-18.

IV. DISCUSSION

In this section, we compare our C-DNN processor with previous hardware architecture [20], which can support both CNN and SNN. This article proposes a higher level design method to achieve higher energy efficiency by combining CNN and SNN. The operational patterns of CNN and SNN are different, and simply mapping SNN onto the previous CNN hardware architecture would result in significant power consumption due to weight memory access. Tianjic [20], with its homogeneous structure, has not addressed this problem. In addition, when Tianjic performs operations in the SNN mode, the multiplier in the MAC unit cannot perform multiplications, leading to reduced utilization and the inability to achieve optimal energy efficiency. Although Tianjic proposes a modeling approach that combines CNN and SNN, it does not optimize by utilizing the characteristic that the energy efficiency superiority relationship between CNN and SNN changes depending on spike sparsity. C-DNN, on the other hand, reveals this tradeoff between CNN and SNN and proposes a novel approach to achieve higher energy efficiency.

Moreover, the most significant difference between Tianjic and C-DNN lies in DNN training. C-DNN calculates the forward delta weight through STDP and predicts the magnitude of the backward gradient based on it. Then, it maximizes training energy efficiency by skipping unnecessary backward gradient computations using the predicted information. Tianjic, on the other hand, only aims to enable both CNN and SNN inference on a homogeneous computational unit and thus lacks the logic to calculate STDP and does not support the proposed training method in C-DNN.

In addition, we further discuss in this section why we can generate sparsity in the backward gradient through forward delta weight. Several recent papers [21], [22], [23] have proposed methods to make backward computations sparse through SNN inference, demonstrating significant energy reduction and speed improvement. Perez-Nieves and Goodman [21] explained that the input trace is utilized to generate sparsity in gradient of supervised SNN training method [BP through time (BPTT)]. Here, the input trace refers to the time difference between the current time step and the time step when the pre-spike occurred previously. It records the history from the time when the input spike occurs. Then, for only neurons with membrane voltage close to the threshold, backward computations are performed by multiplying the input history by the error received from the next layer. During this process, the backward computation result becomes 0 for neurons that do not fire output spikes and neurons that have not fired input spikes for a long time, preventing the transmission of the corresponding error to the previous layer. In other words, based on the intuition that pre-neurons that spiked a long time ago have less relevance to the post-neuron, a small input trace value is multiplied with the gradient computed for BPTT, leading to a smaller weight update. Conversely, pre-neurons that recently spiked are more relevant, so a larger input trace value is multiplied with the gradient, resulting in a larger weight update. While work [21] does not explicitly mention the use of STDP, the input trace used here corresponds to a term calculated when the STDP rule is applied. For the STDP rule, the number of time steps that have passed since the occurrence of the pre-spike and the number of time steps that have passed since the occurrence of the post-spike is recorded. In this process, the first term is used to record how much time has passed since the pre-spike, serving a similar role to the input trace used in [21]. Thus, the training method proposed in this article can be understood as determining the importance of the gradient computed for BP using the input trace term from the STDP rule, following the same logic used in [21], rather than predicting the weight update quantity obtained through STDP. By applying the principles used in [21] directly to BP, we were able to skip 44%/64% of computations during ImageNet training in the BP/GG processes. In addition, both the gradient obtained through BPTT in [21] and the gradient obtained through BP are derived using the supervised rule, indicating a correlation in updating the weights in the same direction. In fact, as seen in [24], there is a strong correlation between the forward pass and backward pass of artificial neural networks (ANNs) and SNN. During the forward pass, the hidden layer values of ANN

and SNN exhibit an extremely strong correlation coefficient, reaching 0.99. Moreover, during the backward pass, assuming the same initial weight initialization, the hidden layer values of the SNN's backward pass, approximated to be differentiable like BPTT, have a high correlation coefficient of 0.75 with ANN's backward pass. Since the training method of ANN, BP, and the training method of SNN, BPTT, have a high correlation, the method in [21] that uses input trace term from STDP to generate sparsity during the BPTT process can also be applied to BP. To demonstrate the effectiveness of this idea, we estimate the correlation between terms that are skipped based on forward information and negligible terms during the backward pass for various datasets. It shows the correlation coefficient of 0.7 for CIFAR-10 training, 0.65 for CIFAR-100 training, and 0.53 for ImageNet training. Moreover, we perform a comparison with two other simple methods (random skipping and magnitude-based pruning). In the case of random skipping for BP and GG operations, only 69.3% accuracy is achieved in ImageNet training of ResNet-50 when skipping 30% and 40% of BP and GG operations, respectively, similar to FDWSG. This means that random methods make accurate training difficult compared to FDWSG. In contrast, the method that generates input sparsity by setting small-magnitude values of error and gradient to zero achieves a high training accuracy of 77.6%, unlike the random method. However, pruning based on magnitude can be applied orthogonally to the FDWSG proposed in this article. This is because FDWSG is a method that predicts the magnitude of the output error/gradient to generate output sparsity, while magnitude-based pruning is a different method that generates input sparsity. When FDWSG is applied together with magnitude pruning, a training accuracy of 76.7% can be achieved, and 51% of BP operations and 65% of GG operations are omitted during training. Furthermore, the skipping threshold value is a hyperparameter, and it is set by the user before training.

Furthermore, in this section, we discuss whether the advantages of the C-DNN architecture persist when processing-in-memory (PIM) is employed for the CNN part. This discussion arises because, in PIM, it is predicted that energy consumption will decrease proportionally to input sparsity, which may raise concerns about the proposed architecture has no advantages compared to CNN PIM. In the case of PIM architecture, power consumed by MAC operations in the bitline (BL) can be reduced due to the decreased activity caused by input sparsity. However, in PIM, the largest power portion in the overall architecture is occupied by the analog-to-digital converter (ADC). Even if sparsity exists, PIM needs to operate the ADC to represent the BL voltage corresponding to the MAC operation results in the digital domain. In other words, although sparsity can reduce BL computation power, the power reduction effect due to increased sparsity cannot be significantly observed because the ADC power, which occupies the most power portion, cannot be reduced. Therefore, even when designing CNN in a PIM architecture, the SNN operation structure, where the computational power decreases proportionally to sparsity, is more efficient under high sparsity conditions.

Moreover, we aim to compare the sparsity exploitation in DNN processors that skip operations based on activation sparsity with SNN processors. In the previous sparsity handling DNN processors, operations are skipped by leveraging sparsity at the data level. These processors omit MAC operations that use inputs with zero values, which do not affect the results, to increase energy efficiency. However, the principle of zero skipping at the data level can be applied to both CNN and SNN. The key point that this article wants to convey is that the energy consumption relationship between CNN and SNN can change based on the magnitude of the non-zero data when processing the non-zero data itself. Similar to how CNN skips operations for zero data and performs MAC operations using non-zero data, SNN can also skip operations for zero data and perform operations on non-zero data through integrate-and-fire behavior. In summary, the benefits of zero-data skipping can be enjoyed in both network domains, and if the spike sparsity resulting from non-zero data is high, the SNN domain will be even more efficient.

Finally, we discuss whether the area sacrificed to increase energy efficiency was justified. Generally, the need to increase cache size arises when performing memory-dominant characteristic operations. A typical CNN exhibits computation-dominant characteristics due to the high reuse of weights. Therefore, to improve the energy efficiency of computations, it is necessary to deploy more PEs to increase the number of operations performed per memory access. For this reason, instead of increasing the L2 cache size, we opt to deploy more PEs and focus on reducing the energy consumption of these elements to achieve higher energy efficiency. The approach adopted to reduce energy consumption is the co-deployment of CNN and SNN PEs, which resulted in an energy efficiency improvement of approximately 25%. Simply increasing the size of the L2 cache will not provide the same energy efficiency gains achieved in the C-DNN processor, as it does not increase the opportunities for data reuse within the CNN structure.

V. CONCLUSION

The C-DNN processor, a hybrid CNN and SNN accelerator, is proposed to overcome the problem of previous hardware and achieve high energy efficiency for inference and training. It can support CNN-SNN workload division by using magnitude-energy tradeoff. The C-DNN processor integrates the CNN-SNN workload allocator and attention module to find a more energy-efficient network domain for each workload. They enable the C-DNN processor to operate at the energy optimal point. Moreover, the SNN PE array with distributed L1 cache can reduce the redundant memory access for SNN processing, resulting in a 42.2%–49.1% reduction. For high energy-efficient training, the FDWSG is proposed to reduce the number of operations for training by 31%–79%. Furthermore, the C-DNN processor integrates the global counter and LDW unit to eliminate power-consuming counters for a forward delta-weight generation. The C-DNN processor is fabricated in 28-nm CMOS technology and the area is 20.25 mm². It consumes 32.3 mW at 50 MHz with 0.7 V,

and 294.1 mW at 200 MHz with 1.1 V, resulting in a 23.5%–33.1% reduction compared to SNN/CNN. Also, it achieves 56% energy reduction for entire DNN training process by the FDWSG generation (@ ImageNet). In conclusion, we propose the C-DNN processor, which is the first complementary combination of CNN and SNN for energy-efficient inference and training with high accuracy, and the fabricated chip shows 77.1% accuracy for ImageNet with the state-of-the-art energy efficiency, 24.5TOPS/W (@ ResNet-50).

REFERENCES

- [1] G. K. Chen, R. Kumar, H. E. Sumbul, P. C. Knag, and R. K. Krishnamurthy, "A 4096-neuron 1M-synapse 3.8 PJ/SOP spiking neural network with on-chip STDP learning and sparse weights in 10 nm FinFET CMOS," in *Proc. IEEE Symp. VLSI Circuits*, Honolulu, HI, USA, Jun. 2018, pp. 255–256.
- [2] S. Li, Z. Zhang, R. Mao, J. Xiao, L. Chang, and J. Zhou, "A fast and energy-efficient SNN processor with adaptive clock/event-driven computation scheme and online learning," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 4, pp. 1543–1552, Apr. 2021, doi: [10.1109/TCSI.2021.3052885](https://doi.org/10.1109/TCSI.2021.3052885).
- [3] J. Park, J. Lee, and D. Jeon, "A 65-nm neuromorphic image classification processor with energy-efficient training through direct spike-only feedback," *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 108–119, Jan. 2020.
- [4] Y. Liu et al., "An 82 nW 0.53 pJ/SOP clock-free spiking neural network with 40 μ s latency for AIoT wake-up functions using ultimate-event-driven bionic architecture and computing-in-memory technique," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 372–374.
- [5] A. Sengupta, Y. Ye, R. Wang, C. Liu, and K. Roy, "Going deeper in spiking neural networks: VGG and residual architectures," *Frontiers Neurosci.*, vol. 13, p. 95, Mar. 2019.
- [6] B. Han, G. Srinivasan, and K. Roy, "RMP-SNN: Residual membrane potential neuron for enabling deeper high-accuracy and low-latency spiking neural network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2020, pp. 13558–13567.
- [7] J. Wu, Y. Chua, M. Zhang, G. Li, H. Li, and K. C. Tan, "A tandem learning rule for effective training and rapid inference of deep spiking neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 1, pp. 446–460, Jan. 2023, doi: [10.1109/TNNLS.2021.3095724](https://doi.org/10.1109/TNNLS.2021.3095724).
- [8] J. Wu et al., "Progressive tandem learning for pattern recognition with deep spiking neural networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 7824–7840, Nov. 2022, doi: [10.1109/TPAMI.2021.3114196](https://doi.org/10.1109/TPAMI.2021.3114196).
- [9] N. Rath and K. Roy, "DIET-SNN: A low-latency spiking neural network with direct input encoding and leakage and threshold optimization," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 6, pp. 3174–3182, Jun. 2023.
- [10] Z. Yan, J. Zhou, and W.-F. Wong, "Near lossless transfer learning for spiking neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 12, pp. 10577–10584.
- [11] S. Kim, S. Kim, S. Hong, S. Kim, D. Han, and H.-J. Yoo, "C-DNN: A 24.5–85.8 TOPS/W complementary-deep-neural-network processor with heterogeneous CNN/SNN core architecture and forward-gradient-based sparsity generation," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 334–336, doi: [10.1109/ISSCC42615.2023.10067497](https://doi.org/10.1109/ISSCC42615.2023.10067497).
- [12] S. Kim, S. Kim, S. Um, S. Kim, J. Lee, and H.-J. Yoo, "SNPU: Always-on 63.2 μ W face recognition spike domain convolutional neural network processor with spike train decomposition and shift-and-accumulation unit," in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, Taipei, Taiwan, Nov. 2022, pp. 2–4.
- [13] S. Woo, J. Park, J. Lee, and I. Kweon, "CBAM: Convolutional Block Attention Module," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 3–19.
- [14] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [15] K. Hirose et al., "Hiddenite: 4K-PE hidden network inference 4D-tensor engine exploiting on-chip model construction achieving 34.8-to-16.0 TOPS/W for CIFAR-100 and ImageNet," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.
- [16] C. Frenkel and G. Indiveri, "ReckOn: A 28 nm sub-mm² task-agnostic spiking recurrent neural network processor enabling on-chip learning over second-long timescales," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.
- [17] D. Han et al., "HNPU: An adaptive DNN training processor utilizing stochastic dynamic fixed-point and active bit-precision searching," *IEEE J. Solid-State Circuits*, vol. 56, no. 9, pp. 2858–2869, Sep. 2021.
- [18] H. Mo et al., "9.2 A 28 nm 12.1 TOPS/W dual-mode CNN processor using effective-weight-based convolution and error-compensation-based prediction," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 146–148.
- [19] J. Park, S. Lee, and D. Jeon, "9.3 A 40 nm 4.81 TFLOPS/W 8 b floating-point training processor for non-sparse neural networks using shared exponent bias and 24-way fused multiply-add tree," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 1–3.
- [20] L. Deng et al., "Tianjic: A unified and scalable chip bridging spike-based and continuous neural computation," *IEEE J. Solid-State Circuits*, vol. 55, no. 8, pp. 2228–2246, Aug. 2020, doi: [10.1109/JSSC.2020.2970709](https://doi.org/10.1109/JSSC.2020.2970709).
- [21] N. Perez-Nieves and D. F. Goodman, "Sparse spiking gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 11795–11808.
- [22] T. C. Wunderlich and C. Pehle, "Event-based backpropagation can compute exact gradients for spiking neural networks," *Sci. Rep.*, vol. 11, no. 1, pp. 1–17, Jun. 2021.
- [23] Y. Zhu, Z. Yu, W. Fang, X. Xie, T. Huang, and T. Masquelier, "Training spiking neural networks with event-driven backpropagation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 30528–30541.
- [24] A. Shrestha, H. Fang, Q. Wu, and Q. Qiu, "Approximating back-propagation for a biologically plausible local learning rule in spiking neural networks," in *Proc. Int. Conf. Neuromorphic Syst.*, Jul. 2019, pp. 1–8, doi: [10.1145/3354265.3354275](https://doi.org/10.1145/3354265.3354275).



Sangyeob Kim (Graduate Student Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2018 and 2020, respectively, where he is currently pursuing the Ph.D. degree.

His current research interests include energy-efficient system-on-chip design, especially focused on deep neural network accelerators, neuromorphic hardware, and computing-in-memory accelerators.



Soyeon Kim (Graduate Student Member, IEEE) received the B.S. and M.S. degrees from the School of Electrical Engineering, Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2019 and 2021, respectively, where she is currently pursuing the Ph.D. degree.

Her research interests include energy-efficient deep learning processor design and intelligent computer vision systems.



Seongyon Hong (Graduate Student Member, IEEE) received the B.S. degree in electrical engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2022, where she is currently pursuing the M.S. degree.

Her research focuses on energy-efficient computing-in-memory accelerators, low-power system-on-chip for machine learning applications, and deep learning algorithms for efficient processing.



Jiwon Choi (Graduate Student Member, IEEE) received the B.S. degree in electrical engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2022, where he is currently pursuing the M.S. degree.

His current research interests include an energy-efficient system-on-chip design, especially focused on accelerators for deep neural networks and hardware-friendly deep learning algorithms.



Sangjin Kim (Graduate Student Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2019 and 2021, respectively, where he is currently pursuing the Ph.D. degree.

His current research interests include intelligent 3-D vision system-on-chip (SoC), low-power SoC design for deep neural network accelerators, and processing-in-memory for efficient machine learning systems.



Donghyeon Han (Graduate Student Member, IEEE) received the B.S., M.S., and Ph.D. degrees from the School of Electrical Engineering, Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2017, 2019, and 2023, respectively.

He is currently a Post-Doctoral Associate at the Massachusetts Institute of Technology (MIT), Cambridge, MA, USA. His Ph.D. research resulted in several publications in ISSCC, SoVC, IEEE

JOURNAL OF SOLID-STATE CIRCUITS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS (TCAS-I), IEEE Micro, AICAS, Hotchips, and Coolchips. His current research interests include low-power system-on-chip design, especially focused on on-device deep neural network training accelerators and hardware-friendly deep learning algorithms.

Dr. Han received the Best Paper Award and the Best Live Demo Award at the 2022 IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS).



Hoi-Jun Yoo (Fellow, IEEE) graduated from the Department of Electronics, Seoul National University, Seoul, South Korea, in 1983. He received the M.S. and Ph.D. degrees in electrical engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 1985 and 1988, respectively.

Dr. Yoo has served as a member of the Executive Committee for the International Solid-State Circuits Conference (ISSCC), the Symposium on Very Large-Scale Integration (VLSI), and the Asian

Solid-State Circuits Conference (A-SSCC), the TPC Chair for the A-SSCC 2008 and the International Symposium on Wearable Computer (ISWC) 2010, the IEEE Distinguished Lecturer from 2010 to 2011, the Far East Chair for the ISSCC from 2011 to 2012, the Technology Direction Sub-Committee Chair for the ISSCC in 2013, the TPC Vice-Chair for the ISSCC in 2014, and the TPC Chair for the ISSCC in 2015. More details are available at <http://ssl.kaist.ac.kr>.